```
# Drop the antisymmetry restriction: A,B,C arbitrary 8x8. Only input: T (same Cayley-Dickson T as script 26).
import numpy as np
exec(open('26_roots_from_form_only.py').read().split("# basis of antisymmetric")[0])
Bs=[]
for p in range(8):
    for q in range(8):
        M=np.zeros((8,8)); M[p,q]=1; Bs.append(M)
cols=[]
for s in range(3):
    for M in Bs:
        r=[np.einsum('ai,ajk->ijk',M,T),np.einsum('bj,ibk->ijk',M,T),np.einsum('ck,ijc->ijk',M,T)][s]
        cols.append(r.ravel())
Ms=np.array(cols).T; u,sv,vt=np.linalg.svd(Ms); r=np.sum(sv>1e-9); null=vt[r:]; d=null.shape[0]
print("full invariance algebra dimension:",d,"| largest dropped sv %.2e, smallest kept %.2e"%(sv[sv<=1e-9].max(),sv[sv>1e-9].min()))
tr=lambda v:[sum(v[s*64+k]*Bs[k] for k in range(64)) for s in range(3)]
L=[tr(v) for v in null]
co=lambda t: null@np.concatenate([m.ravel() for m in t])
cm=lambda X,Y:[X[s]@Y[s]-Y[s]@X[s] for s in range(3)]
print("closure residual %.2e"%max(np.abs(null.T@co(cm(L[a],L[b]))-np.concatenate([m.ravel() for m in cm(L[a],L[b])])).max() for a in range(d) for b in range(d)))
ad=lambda X:np.array([co(cm(X,L[b])) for b in range(d)]).T
ads=[ad(X) for X in L]
Kf=np.array([[np.trace(a@b) for b in ads] for a in ads]); ev=np.linalg.eigvalsh(Kf)
print("Killing form eigenvalues: zero-ish:",np.sum(np.abs(ev)<1e-9),"negative:",np.sum(ev<-1e-9),"positive:",np.sum(ev>1e-9))
# center: elements (aI,bI,cI)
ctr=[(null@np.concatenate([(a*np.eye(8)).ravel() for a in abc for _ in [0]]) if False else None) for abc in []]
# derived algebra dimension: span of all commutators
D=np.array([np.concatenate([m.ravel() for m in cm(L[a],L[b])]) for a in range(d) for b in range(a+1,d)])
print("derived algebra [g,g] dimension:",np.linalg.matrix_rank(D,tol=1e-9))
# do scalar triples preserve T?  (aI,bI,cI) with a+b+c=0
for abc in [(1,-1,0),(1,1,-2),(1,1,1)]:
    r=abc[0]*T+abc[1]*T+abc[2]*T
    print("scalars",abc,"max|invariance residual| = %.1e"%np.abs(r).max())
```